

Overview

JPA entity lifecycle, Hibernate internals, mapping strategies, JPQL, N+1 prevention, two-level caching, locking, Flyway migrations, and performance tuning.

Entity Lifecycle States

new (transient) – no DB identity

managed – tracked by EntityManager (dirty-checked at flush)

detached – session closed; changes not tracked

removed – scheduled for DELETE at flush

Lifecycle callbacks: @PostLoad, @PostPersist, @PostUpdate on Order for audit timestamps.

Key Mapping Annotations

Annotation	Used On	Detail
@MappedSuperclass	AuditableEntity	createdAt, updatedAt, createdBy
@Embedded	Customer.address	Address value object
@ElementCollection	Product.tags	Set<String> in join table
@Converter(autoApply=true)	MoneyConverter	Money record ↔ NUMERIC column
@Enumerated(STRING)	OrderStatus	Never use ORDINAL
@Inheritance(JOINED)	Promotion	PercentagePromotion, FixedAmountPromotion
@Version	Product.stockCount	Optimistic lock; prevents oversell

N+1 Problem & Fixes

Problem	Fix	Class
Order.items is LAZY by default	@EntityGraph({"items", "items.product"})	OrderRepository.findWithItemsById()
N+1 on category per product	JOIN FETCH p.category in JPQL	ProductRepository.findAllWithCategory()
Heavy collection loading	@BatchSize(size=30)	Customer.orders

Test: N1DetectionTest asserts findAllOrders() issues ≤ 2 queries via Hibernate Statistics.

Caching Layers

Level	Provider	Scope	ShopVerse Usage
L1 (first-level)	EntityManager identity map	Per-transaction	Automatic; two findById in same tx → 1 DB hit
L2 (second-level)	Caffeine via hibernate-jcache	Shared across sessions	@Cache(READ_WRITE) on Product, Category
Query Cache	Hibernate	Per-query result	@QueryHint(HINT_CACHEABLE) on CategoryRepository.findAll()

Level	Provider	Scope	ShopVerse Usage
Spring Cache	Redis + Caffeine	Application-level	@Cacheable("products") on ProductService.findById()

Locking Strategies

Type	Annotation	Class	When
Optimistic	@Version int version	Product.stockCount	High-read, rare-write; retry on OptimisticLockException
Pessimistic Write	@Lock(PESSIMISTIC_WRITE)	FlashSaleService.reserveStock()	Flash sale; SELECT FOR UPDATE
Retry	@Retryable(OptimisticLockException.class)	OrderService.placeOrder()	Auto-retry on version conflict

Flyway Migration Sequence

- V1__core_schema.sql – CREATE TABLE for all entities
- V2__add_columns.sql – ALTER TABLE, add discount_amount
- V3__indexes.sql – B-Tree, Partial, Covering, GIN, Expressio
- V4__stored_procedure.sql – sp_complete_order()
- V5__procedures.sql – fn_customer_lifetime_value()
- V6__triggers.sql – audit trigger + status-regression guard
- V7__partitioning.sql – PARTITION BY RANGE(created_at) on or
- V8__add_partitioning_and_triggers.sql